

Vehicle Properties and interaction with Car API

VHAL operates on a property-based system, where each vehicle function is represented as a property ID. These properties are categorized into system properties (managed by Google) and vendor properties (defined by OEMs). The default types.hal file in AOSP defines the standard system properties, while OEMs can extend these using VENDOR property IDs. The types.hal can be found in the aosp path [hardware/interfaces/automotive/vehicle/2.0/types.hal](#)

The vehicle properties however have some structure defined, Properties can be tied to a [zone](#) so that they can be addressed as instances (e.g., left and right instance of the same concept)

For example, properties like PERF_VEHICLE_SPEED(vehicle speed) and DOOR_LOCK(door lock status) are accessed using unique property IDs.

Vehicle properties define the format and structure for car-specific data in types.hal. Each property has:

- A unique Property ID (e.g., INFO_MAKE = 0x0101)
- A data type (String, Integer, Float, etc.)
- A change mode (STATIC, ON_CHANGE, CONTINUOUS)
- An access level (READ, WRITE)

Key Characteristics of Vehicle Properties

1. Unique Property ID

- Every property is identified by a unique integer constant (int propertyId).
- Example: VehicleProperty.PERF_VEHICLE_SPEED (0x11600207), VehicleProperty.IGNITION_STATE (0x11400409)

```
/**
 * Represents ignition state
 *
 * @change_mode VehiclePropertyChangeMode:ON_CHANGE
 * @access VehiclePropertyAccess:READ
 * @data_enum VehicleIgnitionState
 */
IGNITION_STATE = (
    0x0409
    | VehiclePropertyGroup:SYSTEM
    | VehiclePropertyType:INT32
    | VehicleArea:GLOBAL),
```

```
/**
 * Speed of the vehicle
 *
 * The value must be positive when the vehicle is moving forward and negative when
 * the vehicle is moving backward. This value is independent of gear value
 * (CURRENT_GEAR or GEAR_SELECTION), for example, if GEAR_SELECTION is GEAR_NEUTRAL,
 * PERF_VEHICLE_SPEED is positive when the vehicle is moving forward, negative when moving
 * backward, and zero when not moving.
 *
 * @change_mode VehiclePropertyChangeMode:CONTINUOUS
 * @access VehiclePropertyAccess:READ
 * @Unit VehicleUnit:METER_PER_SEC
 */
PERF_VEHICLE_SPEED = (
    0x0207
    | VehiclePropertyGroup:SYSTEM
    | VehiclePropertyType:FLOAT
    | VehicleArea:GLOBAL),
```

2. Property Data Type

- Specifies what type of value the property holds.
- Defined in VehiclePropertyType: INT32, FLOAT, BOOLEAN, STRING, INT32_VEC, FLOAT_VEC, etc.

```
enum VehiclePropertyType : int32_t {  
    STRING          = 0x00100000,  
    BOOLEAN         = 0x00200000,  
    INT32           = 0x00400000,  
    INT32_VEC       = 0x00410000,  
    INT64           = 0x00500000,  
    INT64_VEC       = 0x00510000,  
    FLOAT           = 0x00600000,  
    FLOAT_VEC       = 0x00610000,  
    BYTES           = 0x00700000,  
};
```

3. Access Mode

- Defines how the property can be accessed:
 - READ → Data-only (e.g., vehicle speed)
 - WRITE → Control-only (e.g., HVAC fan speed)
 - READ/WRITE → Both (e.g., HVAC temperature setting)

4. Change Mode

- Describes how property updates are reported:
 - STATIC → Value never changes (e.g., VIN).
 - ON_CHANGE → Reports when value changes (e.g., ignition state).
 - CONTINUOUS → Reports periodically (e.g., speed, RPM).

5. VehicleArea

- List of different supported area types for vehicle properties.
- Some properties may be associated with particular areas in the vehicle. For example, VehicleProperty#DOOR_LOCK property must be associated with a particular door, thus this property must be of the VehicleArea#DOOR area type.

```
enum VehicleArea : int32_t {  
    GLOBAL          = 0x01000000,  
    /** WINDOW maps to enum VehicleAreaWindow */  
    WINDOW          = 0x03000000,  
    /** MIRROR maps to enum VehicleAreaMirror */  
    MIRROR          = 0x04000000,  
    /** SEAT maps to enum VehicleAreaSeat */  
    SEAT            = 0x05000000,  
    /** DOOR maps to enum VehicleAreaDoor */  
    DOOR            = 0x06000000,  
    /** WHEEL maps to enum VehicleAreaWheel */  
    WHEEL           = 0x07000000,  
    MASK            = 0x0f000000,  
};
```

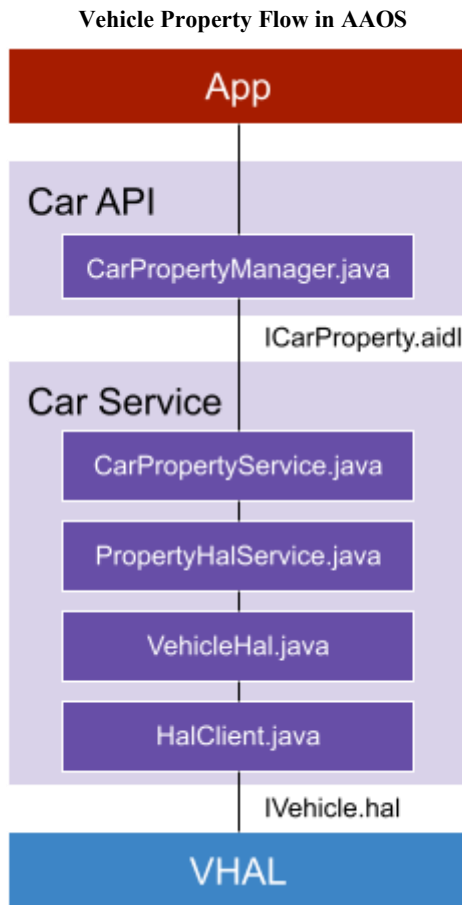
6. VehiclePropertyGroup

- Enumerates property groups.
- Properties declared in AOSP must use SYSTEM flag.
- Properties declared by vendors must use VENDOR flag.

```
enum VehiclePropertyGroup : int32_t {  
    SYSTEM          = 0x10000000,  
    VENDOR          = 0x20000000,  
    MASK            = 0xf0000000,  
};
```

Car API:

- In Android Automotive OS (AAOS), the Car API provides a high-level framework for applications and services to interact with vehicle data and controls. These data points and controls are represented as Vehicle Properties, exposed via the Vehicle HAL (VHAL).
- Car Manager provides domain-specific access to different vehicle subsystems, enabling data retrieval, event handling, and control over key functionalities. The *CarPropertyManager* in the framework layer provides APIs for applications to retrieve and modify these properties.



Summary of Vehicle Property Interaction:

- App (Car API) Uses Car/CarPropertyManager to access properties
- CarPropertyManager API to read/write/register callbacks.
- CarPropertyService → app-facing service (permission checks).
- PropertyHalService → HAL abstraction in CarService (data conversion, caching).
- VehicleHalManager → IPC proxy that talks to the vendor VHAL process.
- Vendor VHAL → actual HAL implementation that interfaces with vehicle hardware.

1. Application / Service

- Uses Car and CarPropertyManager APIs to access vehicle data.
- The app calls CarPropertyManager#registerCallback with the required property ID and update rate.

```
if (getPackageManager().hasSystemFeature(PackageManager.FEATURE_AUTOMOTIVE) && car == null) {  
    car = Car.createCar(this);  
    carPropertyManager = (CarPropertyManager) car.getCarManager(Car.PROPERTY_SERVICE);  
    onCarServiceReady();  
}
```

- Once the **Car API** is initialized, we use CarPropertyManager to check if the PERF_VEHICLE_SPEED property is available:

```
if (carPropertyManager.isPropertyAvailable( VehiclePropertyIds.PERF_VEHICLE_SPEED,  
    VehicleAreaType.VEHICLE_AREA_TYPE_GLOBAL)) {  
    carPropertyManager.registerCallback(  
        callback,  
        VehiclePropertyIds.PERF_VEHICLE_SPEED,  
        CarPropertyManager.SENSOR_RATE_ONCHANGE); // or MAX_UPDATE_RATE_HZ if defined  
} else {  
    Log.e(TAG, "Vehicle speed property not available!");  
}
```

- Registering a Callback for Speed updates. Once the property is available, we register a CarPropertyEventCallback to receive real-time updates whenever the vehicle speed change.

```
CarPropertyEventCallback callback = new CarPropertyEventCallback() {  
    @Override  
    public void onChangeEvent(CarPropertyValue<?> value) {  
        text.setText("Speed: " + value.getValue() + " km/h");  
    }  
  
    @Override  
    public void onErrorEvent(int propertyId, int zone) {  
        text.setText("ERROR: Unable to read speed");  
    }  
};
```

2. Requesting the Required Permissions

- Since vehicle data is restricted, our app needs special privileged permissions to access speed data. These permissions must be defined in AndroidManifest.xml.
- Without these permissions, the app will throw a SecurityException when trying to read PERF_VEHICLE_SPEED.

```
<uses-permission android:name="android.car.permission.CAR_SPEED" />  
<uses-permission android:name="android.permission.READ_VEHICLE_SPEED" />
```

3. Binder IPC to CarService

- These calls go through the AIDL interface ICar to the system-level CarService.

4. CarService → ICarImpl

- CarService delegates the request to ICarImpl, which manages all car services.

5. ICarImpl → CarPropertyService

- ICarImpl routes property-related requests to CarPropertyService, which extends CarServiceBase.

6. CarPropertyService

- The server-side implementation of ICarProperty.aidl.
- Handles client requests, enforces permissions, and forwards to HAL.

```
@Override
public CarPropertyValue getProperty(int propId, int areaId) {
    enforcePermission(propId);           // Security check
    return mPropertyHalService.getProperty(propId, areaId); // Forward request
}
```

7. PropertyHalService

- PropertyHalService is the middle layer inside CarService which connects to the VehicleHalManager (VHAL Binder proxy).
- Implements HAL-specific logic, caching, and filtering before sending data up to CarPropertyService.
- Example :

PropertyHalService.java

```
public CarPropertyValue getProperty(int propId, int areaId) {
    // Call down into VehicleHalManager
    VehiclePropValue value = mVehicleHal.get(propId, areaId);
    return toCarPropertyValue(value);
}

public void setProperty(CarPropertyValue value) {
    VehiclePropValue halValue = toVehiclePropValue(value);
    mVehicleHal.set(halValue);
}
```

- This service converts framework data structures (CarPropertyValue) into HAL structures (VehiclePropValue) and vice-versa.

7. PropertyHalService → Vehicle HAL

- Implements the IVehicleHal abstraction inside CarService.
- Calls the real VHAL binder interface (android.hardware.automotive.vehicle.IVehicle) which is running in a separate process. This is where IPC happens.
- Located in </packages/services/Car/service/src/com/android/car/hal/VehicleHal.java>

```
public VehiclePropValue get(int propId, int areaId) {
    try {
        return mVehicle.get(propId, areaId); // Binder call to real VHAL
    } catch (RemoteException e) {
        throw new IllegalStateException("VHAL not reachable", e);
    }
}
```

7. Vehicle HAL (Vendor Implementation)

- VHAL communicates with the vehicle's hardware via vendor-specific implementations.
- Implemented by the OEM/vendor.
- Talks to the ECUs / CAN bus / sensors to fetch or set real vehicle data.
- It returns property values or sends updates back to CarPropertyService.
- Example:

```
StatusCode VehicleHal::get(const VehiclePropValue& request, VehiclePropValue* outValue) {  
    if (request.prop == VehicleProperty::PERF_VEHICLE_SPEED) {  
        outValue->value.floatValues[0] = readFromCanBus();  
        return StatusCode::OK;  
    }  
    return StatusCode::INVALID_ARG;  
}
```

8. Callback to App

- If a callback was registered, the value is sent back through the same chain to your app via CarPropertyEventCallback and the result is returned as a CarPropertyValue<Float>.

Conclusion

The Car API in AAOS simplifies interaction with complex vehicle data. By using CarPropertyManager and property IDs from VehiclePropertyIds, applications can seamlessly access and control vehicle properties without needing to directly handle HAL or ECU communication.

